

VishwaTech-Labs Python List Methods

□ COLOURFUL HANDWRITTEN-STYLE CLASSROOM NOTES □

Core idea: A list is an ordered, mutable collection. Lists can store multiple values and can contain different data types.

1. LIST BASICS

A **list** is an ordered and mutable collection of items. Lists use square brackets [].

```
skills = ["Python", "AWS", "Docker"]
print(skills)
print(type(skills)) # <class list>
```

Indexing and slicing:

```
skills = ["Python", "AWS", "Docker", "Kubernetes"]
first = skills[0]
last = skills[-1]
part = skills[1:3]
print(first) # Python
print(last) # Kubernetes
print(part) # ['AWS', 'Docker']
```

2. LIST METHODS & USEFUL BUILT-INS

1. append()

Definition: Adds one item to the end of the list. It changes the original list and returns None.

Example — assign the result to a variable:

```
skills = ["Python", "AWS"]
result = skills.append("Docker")
print(skills) # ['Python', 'AWS', 'Docker']
print(result) # None
```

Real-time use: Adding a newly learned skill, task, record or item.

2. extend()

Definition: Adds all items from another iterable to the end of the list.

Example — assign the result to a variable:

```
skills = ["Python", "AWS"]
new_skills = ["Docker", "Kubernetes"]
result = skills.extend(new_skills)
print(skills)
# ['Python', 'AWS', 'Docker', 'Kubernetes']
```

Real-time use: Merging skills, server lists, users or records.

3. insert()

Definition: Inserts an item at a specified index.

Example — assign the result to a variable:

```
skills = ["Python", "Docker"]
result = skills.insert(1, "AWS")
print(skills)
# ['Python', 'AWS', 'Docker']
```

Real-time use: Putting a priority task or record at an exact position.

4. remove()

Definition: Removes the first matching value from the list. Raises ValueError if the value is absent.

Example — assign the result to a variable:

```
servers = ["web01", "web02", "db01"]
result = servers.remove("web02")
print(servers)
# ['web01', 'db01']
```

Real-time use: Removing a known server, user or task.

5. pop()

Definition: Removes and returns an item. If no index is supplied, it removes the last item.

Example — assign the result to a variable:

```
tasks = ["Build", "Test", "Deploy"]
completed_task = tasks.pop()
print(completed_task) # Deploy
print(tasks) # ['Build', 'Test']
```

Real-time use: Processing queues/stacks and completed tasks.

6. clear()

Definition: Removes all items from the list.

Example — assign the result to a variable:

```
cache = ["token1", "token2", "token3"]
result = cache.clear()
print(cache) # []
print(result) # None
```

Real-time use: Clearing temporary data or a working collection.

7. index()

Definition: Returns the index of the first matching value. Raises ValueError if absent.

Example — assign the result to a variable:

```
skills = ["Python", "AWS", "Docker"]
position = skills.index("AWS")
print(position)
# 1
```

Real-time use: Finding the position of a known item.

3. LIST METHODS — SEARCH & ORDER

8. count()

Definition: Counts how many times a value occurs in the list.

Example — assign the result to a variable:

```
status = ["SUCCESS", "FAILED", "SUCCESS", "SUCCESS"]
success_count = status.count("SUCCESS")
print(success_count)
# 3
```

Real-time use: Counting statuses, errors, duplicates or events.

9. sort()

Definition: Sorts the list in place. It returns None.

Example — assign the result to a variable:

```
scores = [70, 95, 60, 88]
result = scores.sort()
print(scores)
# [60, 70, 88, 95]
```

Real-time use: Sorting marks, prices, priorities or names.

10. sort(reverse=True)

Definition: Sorts the list in descending order.

Example — assign the result to a variable:

```
scores = [70, 95, 60, 88]
result = scores.sort(reverse=True)
print(scores)
# [95, 88, 70, 60]
```

Real-time use: Highest score/priority first.

11. sort(key=...)

Definition: Sorts using a key function that defines what should be compared.

Example — assign the result to a variable:

```
names = ["vishwa", "Anu", "kiran"]
result = names.sort(key=str.lower)
print(names)
# ['Anu', 'kiran', 'vishwa']
```

Real-time use: Case-insensitive sorting of names/data.

12. reverse()

Definition: Reverses the list in place. It returns None.

Example — assign the result to a variable:

```
servers = ["web01", "web02", "db01"]
result = servers.reverse()
print(servers)
# ['db01', 'web02', 'web01']
```

Real-time use: Reversing order for processing or display.

13. copy()

Definition: Returns a shallow copy of the list.

Example — assign the result to a variable:

```
skills = ["Python", "AWS", "Docker"]  
backup = skills.copy()  
backup.append("Kubernetes")  
print(skills)  
print(backup)
```

Real-time use: Creating a separate working copy before modification.

4. LIST BUILT-INS — ANALYSIS & VALIDATION

14. len()

Definition: Built-in function that returns the number of items in a list.

Example — assign the result to a variable:

```
servers = ["web01", "web02", "db01"]
server_count = len(servers)
print(server_count)
# 3
```

Real-time use: Counting users, servers, tasks, records, etc.

15. min()

Definition: Built-in function that returns the smallest item.

Example — assign the result to a variable:

```
response_time = [120, 85, 210, 60]
fastest = min(response_time)
print(fastest)
# 60
```

Real-time use: Finding minimum latency, cost or score.

16. max()

Definition: Built-in function that returns the largest item.

Example — assign the result to a variable:

```
response_time = [120, 85, 210, 60]
slowest = max(response_time)
print(slowest)
# 210
```

Real-time use: Finding maximum latency, cost or score.

17. sum()

Definition: Built-in function that adds numeric list values.

Example — assign the result to a variable:

```
monthly_sales = [12000, 15000, 11000]
total_sales = sum(monthly_sales)
print(total_sales)
# 38000
```

Real-time use: Totals for sales, marks, costs or resource counts.

18. sorted()

Definition: Built-in function that returns a new sorted list without changing the original.

Example — assign the result to a variable:

```
scores = [70, 95, 60, 88]
sorted_scores = sorted(scores)
print(scores) # [70, 95, 60, 88]
print(sorted_scores) # [60, 70, 88, 95]
```

Real-time use: When you need both the original and sorted data.

19. in / not in

Definition: Membership operators check whether an item exists in a list.

Example — assign the result to a variable:

```
roles = ["Admin", "Developer", "Auditor"]
is_admin = "Admin" in roles
is_guest = "Guest" not in roles
print(is_admin) # True
print(is_guest) # True
```

Real-time use: Access-control and role checks.

5. REAL-TIME MINI PROJECT — DEVOPS SERVER LIST

```
servers = ["web01", "web02", "db01"]

servers.append("web03")
servers.insert(0, "proxy01")

server_count = len(servers)
is_database_present = "db01" in servers

print("Servers :", servers)
print("Count :", server_count)
print("DB present:", is_database_present)
```

This combines `append()`, `insert()`, `len()` and the `in` operator — a useful way to introduce lists through infrastructure examples.

6. REAL-TIME MINI PROJECT — STUDENT MARKS

```
marks = [78, 92, 65, 88, 95]

total = sum(marks)
highest = max(marks)
lowest = min(marks)
sorted_marks = sorted(marks, reverse=True)

print("Total :", total)
print("Highest :", highest)
print("Lowest :", lowest)
print("Ranking :", sorted_marks)
```

Methods/built-ins: `sum()`, `max()`, `min()`, `sorted()`.

7. IMPORTANT DIFFERENCE — `sort()` vs `sorted()`

```
scores = [70, 95, 60, 88]

result1 = scores.sort()
print(scores) # original list changed
print(result1) # None

scores = [70, 95, 60, 88]
result2 = sorted(scores)
print(scores) # original unchanged
print(result2) # new sorted list
```

Remember: `sort()` changes the existing list; `sorted()` creates and returns a new sorted list.

8. IMPORTANT DIFFERENCE — `append()` vs `extend()`

```
skills = ["Python"]
skills.append(["AWS", "Docker"])
print(skills)
# ["Python", ["AWS", "Docker"]]

skills = ["Python"]
skills.extend(["AWS", "Docker"])
print(skills)
# ["Python", "AWS", "Docker"]
```

Remember: `append()` adds one object as one item. `extend()` adds items from an iterable one by one.

9. PRACTICE LABS

① Create a list of 5 cloud technologies.
② Add Kubernetes using `append()`.
③ Add several tools using `extend()`.
④ Insert Terraform at position 1.
⑤ Remove one tool using `remove()`.
⑥ Use `pop()` to process the last task.
⑦ Count repeated values using `count()`.
⑧ Find a value's position using `index()`.
⑨ Sort marks ascending and descending.
⑩ Find total, highest and lowest marks.

Check whether 'AWS' exists using in.
□ Build a server inventory mini-project.

10. QUICK REFERENCE

Operation / Method	What it does
append(x)	Adds x at the end
extend(iterable)	Adds all items from iterable
insert(i, x)	Adds x at index i
remove(x)	Removes first matching x
pop([i])	Removes and returns item
clear()	Removes all items
index(x)	Finds first position
count(x)	Counts occurrences
sort()	Sorts original list
reverse()	Reverses original list
copy()	Creates shallow copy
len(list)	Number of items
sorted(list)	Returns new sorted list
min(list)	Smallest value
max(list)	Largest value
sum(list)	Total of numeric values
x in list	Membership check
x not in list	Non-membership check

Teaching formula: INPUT → METHOD → VARIABLE → OUTPUT → REAL-TIME USE.

VishwaTech-Labs • Python Training Notes